QuadrupolarDFT

User Manual

Ab initio electric-field gradients and finite-temperature NQR parameters

QuadrupolarDFT is the electric-field-gradient (EFG) workspace within the MRSpinDynamics repository. It converts first-principles EFG tensors into nuclear quadrupolar coupling constants, asymmetry parameters, and zero-field NQR transition frequencies, and corrects those 0 K static quantities to a finite temperature through harmonic vibrational averaging driven by a finite-displacement ABINIT DFPT workflow.

MRSpinDynamics project • June 2026

Contents

1 Scope and Physical Background

QuadrupolarDFT addresses one link in the magnetic-resonance modelling chain: it turns an *ab initio* electronic-structure result into the parameters that control a nuclear quadrupole resonance (NQR) experiment.

Readers new to electronic-structure theory may want to start with the companion `docs/DFT_for_Spin_Dynamics_Beginners` which builds up the background this manual assumes — the SCF loop, k-points and basis sets, PAW versus all-electron treatments, and phonons — from a spin-dynamics standpoint.

A nucleus with spin $I \geq 1$ carries an electric quadrupole moment Q that couples to the electric-field-gradient (EFG) tensor $V_{ij} = \partial^2 V / \partial x_i \partial x_j$ produced by the surrounding charge distribution. The EFG is symmetric and, away from the nucleus, traceless (Laplace's equation), so in its principal-axis system it is described by two numbers: the largest-magnitude component V_{zz} and the asymmetry parameter

$$\eta = \frac{V_{xx} - V_{yy}}{V_{zz}}, \qquad |V_{zz}| \geq |V_{yy}| \geq |V_{xx}|, \qquad 0 \leq \eta \leq 1. \tag{1.1}$$

The quadrupolar coupling constant follows from V_{zz} and the nuclear quadrupole moment,

$$C_{\mathrm{Q}} = \frac{e\,Q\,V_{zz}}{h}, \tag{1.2}$$

and the zero-field NQR transition frequencies follow from C_{Q}, η and the spin I by diagonalizing the quadrupole Hamiltonian

$$\frac{H_{\mathrm{Q}}}{h} = \frac{C_{\mathrm{Q}}}{4I(2I-1)} \left[3I_z^2 - I(I+1) + \eta \left(I_x^2 - I_y^2 \right) \right]. \tag{1.3}$$

For spin $I = 1$ (e.g. ^{14}N) this gives the familiar triplet

$$\nu_{\pm} = \tfrac{3}{4} C_{\mathrm{Q}} \left(1 \pm \tfrac{\eta}{3} \right), \qquad \nu_0 = \nu_+ - \nu_- = \tfrac{1}{2} C_{\mathrm{Q}}\, \eta. \tag{1.4}$$

1.1 Why a finite-temperature correction is needed

A static DFT calculation evaluates V_{ij} at a single, fixed nuclear configuration at 0 K. Measured NQR frequencies, however, are taken at finite temperature and are strongly temperature dependent: in molecular crystals the lines typically *fall* as temperature rises (the Bayer law). Two physically distinct effects separate the static DFT number from the measurement:

1. **Thermal expansion / geometry.** The lattice constants and internal coordinates at temperature T differ from the relaxed 0 K cell. This is a static effect: it is captured by evaluating the EFG on a temperature-appropriate structure.
2. **Vibrational averaging.** The nucleus samples a thermal distribution of positions through the phonons. The measured EFG is the tensor averaged over that motion, including zero-point motion, which is present even at 0 K. For molecular crystals this dynamic term usually dominates: low-frequency librations reorient the EFG principal axes and reduce the line frequencies.

QuadrupolarDFT provides the static layer (Chapter 3) and a harmonic treatment of the vibrational term (Chapters 4–5).

2 Installation and Conventions

2.1 Installation

The package depends only on NumPy. From the `QuadrupolarDFT/` directory:

```
python -m pip install -e .
python -m pip install -e ".[dev]"     # adds ruff for linting
python -m unittest discover -s tests
```

On a system whose Python is externally managed (PEP 668), the package can also be used without installation by setting `PYTHONPATH` to the `src/` directory, since NumPy is the only requirement:

```
export PYTHONPATH="$PWD/src"
python3 examples/abinit/efg_temperature.py --help
```

2.2 Tensor and unit conventions

`EFGTensor` stores the symmetric tensor in SI units (V/m^2) and sorts the principal components as $|V_{zz}| \geq |V_{yy}| \geq |V_{xx}|$. It accepts input in ABINIT atomic units (the default), in SI, or in the 10^{21} V/m^2 scale, and exposes `vzz_si` and `eta`. Tensor averaging (`average_tensors`) operates on the tensor components and only then diagonalizes the result, so that asymmetry information survives the average.

2.3 Link to the spin-dynamics simulator

The companion `spin_dynamics` package parameterizes a quadrupolar site by its $\eta = 0$ transition frequency ν_{Q}. This is related to C_{Q} by the prefactor of Eq. (1.3):

$$\nu_{\mathrm{Q}} = C_{\mathrm{Q}} \cdot \frac{d}{4I(2I-1)} = \begin{cases} \frac{3}{4}\, C_{\mathrm{Q}} & I = 1, \\ \frac{1}{2}\, C_{\mathrm{Q}} & I = \frac{3}{2}, \end{cases} \tag{2.1}$$

with $d = 3$ for spin 1 and $d = 6$ for spin 3/2. The cross-project `mr_integration` layer uses this mapping to feed DFT parameters into the simulator and to compare predictions against the measured NQR database.

3 Static EFG Workflow

The static path produces C_{Q}, η and the NQR lines from an ABINIT EFG calculation.

1. `format_abinit_efg_block` emits the `nucefg`/`quadmom` input lines that request an EFG calculation (PAW datasets are required; norm-conserving pseudopotentials are not suitable).
2. ABINIT is run on the input (see `examples/abinit/run_nano2_efg_wsl.sh`).
3. `parse_abinit_efg` parses the `Electric Field Gradient Calculation` blocks of the output into `AbinitEFGRecord` objects, one per atom, carrying C_{Q}, η, the principal values/axes, and the total EFG tensor.
4. `coupling_constant_hz` and `nqr_frequencies_hz` convert a tensor and a nuclear quadrupole moment into C_{Q} and the unique zero-field transition frequencies (Eqs. (1.2)–(1.3)).

`examples/parse_abinit_efg.py` demonstrates parsing an output file. The EFG components, not the total energy, must be converged against `ecut`, `pawecutdg` and the k-point mesh.

4 Finite-Temperature EFG

4.1 Harmonic vibrational averaging

Expanding the EFG tensor to second order in the mass-weighted phonon normal coordinates Q_k and averaging over the harmonic distribution gives

$$\langle V_{ij}\rangle(T) = V_{ij}^{\mathrm{eq}} + \tfrac{1}{2}\sum_k \frac{\partial^2 V_{ij}}{\partial Q_k^2}\,\langle Q_k^2\rangle(T), \tag{4.1}$$

where the mean-square amplitude of mode k (angular frequency ω_k) is

$$\langle Q_k^2\rangle(T) = \frac{\hbar}{2\omega_k}\coth\!\left(\frac{\hbar\omega_k}{2k_{\mathrm{B}}T}\right). \tag{4.2}$$

The $\coth$ factor equals $2n(\omega_k,T)+1$ with n the Bose–Einstein occupation. It tends to 1 as $T\to 0$ (pure zero-point motion — which is why the static EFG is never exactly what is measured) and to $2k_{\mathrm{B}}T/\hbar\omega_k$ in the classical high-temperature limit. These quantities live in `quadrupolar_dft.thermal`.

Tensor-frame averaging. The average in Eq. (4.1) is taken on the tensor components in a fixed crystal frame; the result is diagonalized only afterwards. Librational motion reorients the principal-axis system, so averaging V_{zz} or η directly would discard exactly that effect. As a consequence η is itself temperature dependent. `thermally_averaged_efg` performs the average and removes any residual trace (the EFG is traceless by Laplace's equation; the finite-difference curvatures of Section 5.1 amplify rounding noise into a spurious trace, which is projected out). `efg_temperature_sweep` returns $C_{\mathrm{Q}}(T)$, $\eta(T)$ and the lines at each requested temperature.

4.2 The analytic Bayer limit

Where per-mode EFG curvatures are not available, the temperature *form* can be validated directly against a measured line series with the single-libration Bayer–Kushida model

$$\nu(T) = \nu_0\left[1 - a\,\coth\!\left(\frac{\hbar\omega}{2k_{\mathrm{B}}T}\right)\right], \tag{4.3}$$

where a is a small dimensionless librational amplitude. `fit_bayer_single_mode` fits (ν_0, a, ω) to measured $\nu(T)$ data with a NumPy-only routine: for a fixed ω the model is linear in ν_0 and $\nu_0 a$, so a grid scan over ω with a linear least-squares solve at each point suffices.

5 The DFPT Finite-Displacement Workflow

The mode curvatures $\partial^2 V_{ij}/\partial Q_k^2$ in Eq. (4.1) come from displacing the structure along each phonon mode and recomputing the EFG. Because the DFT runs are expensive and run outside the package, the workflow separates into three stages with a calculation between each, preceded by an optional structure-relaxation stage (Stage 0) that puts the geometry at an energy minimum first. The CLI `examples/abinit/efg_temperature.py` drives all of them.

5.1 Normal-coordinate convention

A phonon eigenvector $\boldsymbol{\varepsilon}_k$ is mass-weighted and normalized ($\sum |\varepsilon_k|^2 = 1$). A normal-coordinate step δQ displaces atom i (mass m_i) by

$$\Delta \mathbf{r}_i = \delta Q \, \frac{\boldsymbol{\varepsilon}_{k,i}}{\sqrt{m_i}}, \tag{5.1}$$

and the curvature is the central difference of the EFG tensors at $0, \pm\delta Q$,

$$\frac{\partial^2 V_{ij}}{\partial Q_k^2} \approx \frac{V_{ij}(+\delta Q) - 2V_{ij}(0) + V_{ij}(-\delta Q)}{\delta Q^2}. \tag{5.2}$$

These units are consistent with $\langle Q_k^2 \rangle$ from Eq. (4.2), so the product in Eq. (4.1) is an EFG. `normal_coordinate_step_si` chooses δQ from a target maximum Cartesian displacement (default 0.05 Å).

5.2 Stage 0 — structure relaxation (optional)

A static EFG is only physical at an energy minimum. On an unrelaxed geometry the residual forces appear as imaginary modes at Γ and bias the equilibrium η, so relaxing first is the single largest source of quantitative error in the chain. `efg_temperature.py relax` strips the EFG keywords from a converged static input and prepends a BFGS ionic-relaxation block (`ionmov 2`, `optcell 0`, `tolmxf`); `run_relax_wsl.sh` runs ABINIT. `efg_temperature.py relax-collect` then reads the relaxed geometry from the output footer (`-outvars: echo values of variables after computation`, the same block abipy reads) and writes a relaxed static EFG input.

```
python3 examples/abinit/efg_temperature.py relax \
    --base examples/abinit/nano2_efg.abi --out runs/nano2_relax
bash examples/abinit/run_relax_wsl.sh runs/nano2_relax
python3 examples/abinit/efg_temperature.py relax-collect \
    --base examples/abinit/nano2_efg.abi --abo runs/nano2_relax/relax.abo \
    --out runs/nano2_relax
```

The product, `relaxed.abi`, is an ordinary static EFG input (EFG keywords intact) at the relaxed geometry; pass it as `-base` to Stages 1–2 in place of the hand-written input. The relaxation is internal-coordinates-only: the cell is held at the input value (`optcell 0`). For molecular crystals this is usually preferable to a full cell relaxation, which under GGA tends to over-expand the lattice and can worsen the EFG relative to the measured geometry; full cell relaxation (the quasi-harmonic thermal-expansion path) is a planned extension.

5.3 Stage 1 — phonons

`efg_temperature.py phonon` writes a two-dataset ABINIT input (ground state plus a Γ-point DFPT response) and the matching anaddb input, derived from a converged static EFG input. `run_phonon_wsl.sh` runs ABINIT, selects the DFPT derivative database (the largest `*_DDB`), and runs anaddb to produce the phonon frequencies and eigenvectors.

```
python3 examples/abinit/efg_temperature.py phonon \
    --base examples/abinit/nano2_efg.abi --out runs/nano2_ph
bash examples/abinit/run_phonon_wsl.sh runs/nano2_ph
```

The generated DFPT input is a starting template. Two settings are required for PAW + GGA phonons and are emitted automatically: `iscf2 7` (the response driver requires `iscf` ≤ 9) and `pawxcdev 0` (the GGA exchange correlation kernel is not implemented in the development scheme). Converge the tolerances and k-mesh for production use.

5.4 Stage 2 — displaced EFG runs

`efg_temperature.py displace` parses the anaddb eigenvectors (`parse_anaddb_modes`; acoustic and imaginary modes are dropped, and modes are paired with frequencies by mode number so that gaps in the eigenvector output do not misalign them), generates one displaced EFG input per $\pm$ mode plus a manifest, and writes positions as fractional `xred` coordinates. `run_finite_displacement_wsl.sh` runs ABINIT on every input.

```
python3 examples/abinit/efg_temperature.py displace \
    --base examples/abinit/nano2_efg.abi --anaddb runs/nano2_ph/anaddb.out \
    --target 2 --max-modes 6 --out runs/nano2_disp
bash examples/abinit/run_finite_displacement_wsl.sh runs/nano2_disp
```

`-target` is the 0-based index of the resonant nucleus; `-max-modes` restricts the calculation to the lowest-frequency librations, which dominate the temperature dependence. If the anaddb eigenvector layout differs from the parser's expectation, `-modes modes.json` supplies frequencies and eigenvectors directly.

5.5 Stage 3 — collect

`efg_temperature.py collect` parses the displaced EFG outputs, central differences the mode curvatures, runs the temperature sweep, and reports $C_{\mathrm{Q}}(T)$, $\eta(T)$, the lines, and $\mathrm{d}\nu/\mathrm{d}T$.

```
python3 examples/abinit/efg_temperature.py collect \
    --workdir runs/nano2_disp --temperatures 0,77,150,300 --quadmom 0.02044
```

5.6 Running ABINIT in parallel

The phonon and EFG runs dominate the wall-clock cost, and ABINIT (MPI-built) parallelizes them efficiently over k-points. All the runner scripts source a small shared helper, `examples/abinit/abinit_parallel.sh`, and honor the `ABINIT_NP` environment variable:

- `ABINIT_NP` unset (or `1`) — serial.
- `ABINIT_NP=`N — launch `mpirun -np` N `abinit`.
- `ABINIT_NP=auto` — choose N automatically (see below) and fall back to serial if no MPI runtime is found.

With the default k-point parallelism (`paral_kgb 0`), efficiency is good when the number of processes divides the number of k-points in the IBZ. The helper function `abinit_optimal_np` encodes this: it returns the largest divisor of n_{kpt} that is $\leq$ the core count. For the $NaNO_2$ cell ($n_{\mathrm{kpt}} = 36$) on a 20-core machine that is $N = 18$ (two k-points per process), which scales near-ideally ($\sim 18\times$). `auto` detects n_{kpt} from an existing `.abo` or a quick `abinit -dry-run`, and the core count from `nproc`. Override the launcher with `ABINIT_MPIRUN` (e.g. `mpiexec`, or `mpirun -oversubscribe`). The driver `nano2_relaxation_study.sh` uses `auto` by default; pass `-np` N to pin a value or `-np 1` to force serial.

Parallelization changes only how ABINIT is launched, not the inputs, so it leaves all computed quantities (and any relaxed-vs-unrelaxed comparison) unchanged.

5.7 Convergence of η and the asymmetry gap

The coupling $C_{\mathrm{Q}} \propto V_{zz}$ tracks the largest EFG eigenvalue and converges quickly, but the asymmetry $\eta = (V_{xx} - V_{yy})/V_{zz}$ is a normalized *difference* of the two smaller eigenvalues. It therefore converges much more slowly with the PAW double-grid cutoff `pawecutdg` and the k-mesh `ngkpt`, and a calculation can reproduce C_{Q} while still reporting a too-axial (low) η simply because the transverse tensor components are not yet converged. `examples/abinit/efg_convergence.py` isolates that effect: it sweeps `ecut`, `pawecutdg`, and `ngkpt` one knob at a time around the baseline read from a base input, then tabulates η (and C_{Q}) per knob with successive deltas and a converged flag.

```
PYTHONPATH=src python3 examples/abinit/efg_convergence.py generate \
    --base examples/abinit/nano2_efg.abi --target 2 \
    --ecut 25,30,35 --pawecutdg 50,60,80,100 --ngkpt 4,6,8 \
    --out runs/nano2_conv
bash examples/abinit/run_convergence_efg_wsl.sh runs/nano2_conv
PYTHONPATH=src python3 examples/abinit/efg_convergence.py collect \
    --workdir runs/nano2_conv --out runs/nano2_conv/convergence.md \
    --csv runs/nano2_conv/convergence.csv
```

Each knob's ladder shares one baseline job (only off-baseline values are staged as extra runs), so the baseline is computed once and anchored across all three sweeps. `run_convergence_efg_wsl.sh` runs each job in a WSL-native scratch directory (a `mktemp` dir under `$TMPDIR`, overridable with `ABINIT_SCRATCH_DIR`) and copies only `<name>.abo` back into the workdir. This matters on OneDrive-synced `/mnt/c` checkouts, where ABINIT scratch and `.abo` I/O is slow and OneDrive file locks can stall a run so that no `.abo` ever appears.

$NaNO_2$ result. For the ^{14}N site the sweep is decisive: η is flat to $\sim 10^{-4}$ across `pawecutdg` ($50 \to 100$ Ha) and `ngkpt` ($4^3 \to 8^3$, n_{kpt} up to 260), while `ecut` moves it only slightly and *away* from experiment (knee near 30 Ha). Convergence — and, because the inputs run with `nsym 1`, symmetry pinning — is therefore ruled out as the cause of the low η. The geometry is the dominant lever instead:

geometry	η	C_{Q} (MHz)
hand-built starter	0.043	-5.17
ICSD 82857 (experimental)	0.112	-5.03
experiment (77–300 K NQR)	~ 0.38	± 5.49

The experimental cell raises η by $\sim 2.6\times$ over the starter geometry but still under-predicts the measured value. The cause of that residual was pinned down with an all-electron cross-check.

All-electron cross-check (Elk): the gap is a PAW artifact. The transverse EFG components that set η are a near-nucleus quantity, so they are sensitive to how the pseudopotential reconstructs the core region. Repeating the calculation with the *same* PBE functional and the *same* ICSD geometry in the all-electron full-potential LAPW code Elk — the reference method for EFGs — gives a very different η:

method (PBE, ICSD geometry)	η	$\lvert C_{\mathrm{Q}}\rvert$ (MHz)
experiment (77–300 K NQR)	~ 0.38	5.49
ABINIT — PAW (Pseudodojo)	0.112	5.03
Elk — all-electron LAPW	≈ 0.333	5.9

All-electron PBE recovers $\eta \approx 0.333$ (converged over `rgkmax` $7 \to 8$ and the k-mesh, $18 \to 48$ points), next to the measured 0.38, whereas the PAW value sits $\sim 3\times$ lower. Same functional, same geometry: the long-standing η under-prediction is a **PAW/pseudopotential artifact** — the Pseudodojo PBE nitrogen dataset under-represents the near-nucleus transverse EFG — *not* a functional or dynamics deficiency. (Harmonic vibrational averaging, the finite-temperature workflow above, in fact *lowers* η, so dynamics does not close the gap upward.) The ABINIT-side fix is a harder / EFG-tuned nitrogen PAW dataset; a meta-GGA or hybrid functional is not needed. The Elk cross-check workflow — input, an `EFG.OUT` parser, and an MPI-parallel runner with a convergence watchdog — is in `examples/elk/` (see its `README.md`); the original ranked plan is in `docs/eta_accuracy_improvement.md`.

6 Finite-Strain EFG Drive Couplings

Piezoelectric NQR drive needs a different DFT derivative from the finite-temperature workflow. Instead of displacing atoms along optical phonon normal modes, the experiment applies a macroscopic strain field through the piezoelectric crystal. To first order,

$$V_{ij}(\boldsymbol{\epsilon}) \simeq V_{ij}(0) + \sum_{\mu} \frac{\partial V_{ij}}{\partial \epsilon_{\mu}} \epsilon_{\mu},$$

where $\mu \in \{xx, yy, zz, yz, xz, xy\}$ is the symmetric strain component in Voigt notation. The transition-drive coefficient is obtained by projecting $\partial V_{ij}/\partial \epsilon_{\mu}$ into the eigenbasis of the equilibrium quadrupolar Hamiltonian. This is the quantity used by the piezoelectric drive/detection model in PythonSpinDynamics.

6.1 Glycine polymorph guard

The bundled glycine structure is CCDC 189379, space group `P 21`. This non-centrosymmetric polymorph is symmetry-allowed to be piezoelectric, whereas centrosymmetric glycine polymorphs are not useful for an electric-field-driven piezoelectric transducer. The CLI therefore performs a CIF metadata check before writing jobs:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py check
```

For CCDC 189379, the expanded ABINIT cell contains two symmetry-related nitrogen sites. The zero-based target indices printed by `prepare-base` are `1` and `11`; the example below uses index `1`.

6.2 Static and strained ABINIT jobs

The strain workflow begins by preparing a static EFG input directly from the CIF. On Ubuntu's packaged ABINIT 9.10.4 PAW set, the glycine H/C/N/O XML files are in `Pseudodojo_paw_pw_standard`. Those PAW spheres overlap on short X–H bonds, so the exploratory run used `pawovlp 25`; production calculations should replace this with smaller-radius PAW datasets or a validated pseudopotential choice.

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py prepare-base \
  --out runs/glycine_static/glycine_efg.abi \
  --pawovlp 25

bash examples/abinit/run_static_efg_wsl.sh \
  runs/glycine_static/glycine_efg.abi
```

After the static input is validated, the generator writes the equilibrium plus central-difference strained structures:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py generate \
  --base runs/glycine_static/glycine_efg.abi \
  --target-atom-index 1 \
  --out runs/glycine_strain

bash examples/abinit/run_strain_efg_wsl.sh runs/glycine_strain
```

The collection step parses each `.abo`, finite-differences the target nucleus EFG tensor, diagonalizes the equilibrium spin-1 Hamiltonian, and writes JSON/CSV coupling tables:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py collect \
  --workdir runs/glycine_strain \
  --quadmom 0.02044 \
  --strain-peak 1e-5 \
  --json runs/glycine_strain/couplings.json \
  --csv runs/glycine_strain/couplings.csv
```

6.3 First-pass glycine result

The exploratory glycine calculation completed all 13 jobs (equilibrium plus $\pm$ strains for six components). The equilibrium EFG at the target nitrogen was

$$(V_{xx}, V_{yy}, V_{zz}) = (-1.9249, -1.2972, 3.2221) \times 10^{21}\ \mathrm{V/m^2}, \qquad \eta = 0.1948,$$

corresponding to $C_\mathrm{Q} = 1.5925\,\mathrm{MHz}$ for ^{14}N. The NQRDatabase glycine amine-^{14}N entry is $C_\mathrm{Q} = 1.193\,\mathrm{MHz}$ and $\eta = 0.528$, with observed lines at 737 and 1052 kHz. Thus the DFT run captures the MHz scale but makes the tensor too axial; absolute frequencies are not yet production quality.

Transition	Best strain	Frequency (kHz)	Coupling (MHz/strain)	Rabi at 10^{-5} strain (Hz)
$1 \to 2$	xz	155.124	1.456	7.28
$0 \to 1$	yy	1116.801	1.844	9.22
$0 \to 2$	xz	1271.924	2.189	10.95

Table 6.1: Strongest projected quadrupolar drive coefficients from the first-pass glycine finite-strain calculation. The Rabi rate assumes a peak acoustic strain of 10^{-5}.

For the experimental two-line glycine model in PythonSpinDynamics, the strongest DFT coefficients are mapped onto the observed lower and upper lines as practical drive estimates: the lower 737 kHz line uses the yy-driven $0 \to 1$ coefficient ($1.844\,\mathrm{MHz/strain}$), and the upper 1052 kHz line uses the xz-driven $0 \to 2$ coefficient ($2.189\,\mathrm{MHz/strain}$). This keeps the detection model tied to measured frequencies while using DFT-scale electric-field-gradient derivatives.

Reliability checklist. Before using these couplings as design numbers, repeat the workflow with converged `ecut`, `pawecutdg`, and `ngkpt`; relax at least the hydrogen positions or use a neutron-quality structure; verify the symmetry-related nitrogen index `11`; and remove or quantify the effect of `pawovlp 25`.

7 Worked Example: $NaNO_2$ ^{14}N

Sodium nitrite is the reference case for the workspace: its ^{14}N parameters appear in the DFT runs, the spin-dynamics simulator, and the measured NQR database. A six-mode finite-displacement calculation on the starter geometry produces the temperature dependence in Table 7.1.

T (K)	C_Q (MHz)	η	lines (MHz)
static	-5.173	0.043	0.11, 3.82, 3.94
0	-4.963	0.016	0.04, 3.70, 3.74
77	-4.927	0.014	0.03, 3.68, 3.71
150	-4.816	0.002	0.00, 3.61, 3.61
300	-4.534	0.033	0.08, 3.36, 3.44

Table 7.1: Finite-temperature ^{14}N EFG of $NaNO_2$ from a six-mode harmonic average on the starter geometry. The slope of the two strong lines is $\mathrm{d}\nu/\mathrm{d}T \approx -1.0$ to -1.1 kHz/K over 0–300 K.

The result reproduces the expected physics with no fitting: $|C_Q|$ and the lines decrease smoothly with temperature (the Bayer sign), the 0 K value already lies below the static one by about 4% (zero-point motion), and η shifts with temperature. The slope, ~ -1.0 kHz/K, has the correct sign and order of magnitude relative to the measured coefficients of -1.3 kHz/K (ν_-) and -2.2 kHz/K (ν_+).

Caveat. The numbers above are for the *unrelaxed* starter geometry: its static $\eta = 0.04$ disagrees with the experimental 0.38, the two strong lines come out near-degenerate (~ 3.7 MHz) rather than well split ($3.60/4.64$ MHz), and the phonon spectrum contains imaginary modes. The pipeline is therefore validated as physically sensible, but quantitative agreement requires relaxing the structure to an energy minimum first — run Stage 0 (§5.2) and pass the resulting `relaxed.abi` as the base input to remove the imaginary modes and correct η before interpreting the absolute numbers.

Runnable comparison. `examples/abinit/nano2_relaxation_study.sh` automates exactly this check: it runs the full temperature workflow on both the unrelaxed and the relaxed geometry and writes a side-by-side report (static EFG, the sweep, and $\mathrm{d}\nu/\mathrm{d}T$) scored against the measured $NaNO_2$ ^{14}N reference. See `examples/abinit/README_relaxation_study.md` for the prerequisites and the one-command invocation.

Relaxation outcome. Running this on the starter geometry confirms the pipeline behaves correctly and is qualitatively improved by relaxation, but also exposes its current limit. Relaxing the internal coordinates removes all imaginary modes (the spectrum becomes dynamically stable, lowest libration $\sim 44\,\mathrm{cm}^{-1}$) and fixes the sign of every $\mathrm{d}\nu/\mathrm{d}T$, and it moves the asymmetry and the line splitting toward experiment — but only about a third of the way ($\eta : 0.04 \to 0.14$ against the measured 0.38; strong-line splitting $0.11 \to 0.37$ MHz against ~ 1.04 MHz). Because the cell was held fixed at the hand-entered starter (`optcell 0`, unconverged `ecut`/`pawecutdg`/ `ngkpt`), the residual error sits in the *equilibrium* geometry, not the vibrational average. Quantitative agreement therefore needs a converged calculation on an experimental structure (e.g. the ICSD 82857 input under `structures/NaNO2/generated/`), and, for the full slope magnitude, anharmonic (AIMD/PIMD) averaging near the ferroelectric transition.

CIF validation outcome. The bundled $NaNO_2$ CIFs now have a two-stage check. `examples/check_nano2_cifs.py` first ranks the available structures by phase, temperature, occupancy and crystallographic metadata; for the room-temperature ferroelectric NQR reference, ICSD 82857 is the best structural target. `examples/abinit/nano2_cif_efg_validation.sh` then stages the top full-occupancy CIFs, runs static

EFG calculations through the MPI-aware ABINIT wrapper, and compares the resulting ^{14}N lines against the measured $NaNO_2$ reference. In the first validation sweep the best simulated line agreement came from ICSD 15400, but the full-occupancy ferroelectric structures all gave the same qualitative answer: $|C_Q|$ was close, while η remained too small ($\eta \simeq 0.11$–0.12 versus the measured 0.38).

Interpretation. The $NaNO_2$ asymmetry error is therefore not fixed by choosing a different bundled CIF. The dominant missing ingredient is more likely the local equilibrium tensor itself: convergence of the EFG tensor components, cell/internal-coordinate relaxation against the chosen functional and PAW data, possible functional or PAW-dataset dependence, and eventually anharmonic or snapshot averaging close to the ferroelectric transition. High-temperature `I m m m` CIFs should also be treated carefully: the available entries are partial-occupancy/disordered models, and a literal static expansion can place split sites nearly on top of each other, causing ABINIT PAW sphere-overlap aborts. Use those only after constructing an ordered approximant, not as ordinary static EFG inputs.

8 Validation Against the Measured Database

The cross-project `mr_integration` layer closes the loop against the NQRDatabase. `measured_temperature_coefficients` reads the stored $\mathrm{d}\nu/\mathrm{d}T$ for a compound and isotope; `compare_temperature_coefficients` matches a predicted set of (frequency, $\mathrm{d}\nu/\mathrm{d}T$) pairs — from either a `efg_temperature_sweep` (via `slopes_from_temperature_points`) or a Bayer fit — to the measured coefficients by frequency, so that a DFT temperature dependence can be checked against experiment. The same layer also validates the static (C_Q, η) against measured line positions and supplies the $C_\mathrm{Q} \leftrightarrow \nu_\mathrm{Q}$ conversion used by the simulator.

9 Public API Reference

9.1 quadrupolar_dft.tensors / quadrupolar / abinit

- `EFGTensor`, `average_tensors` — tensor conventions and frame-correct averaging.
- `coupling_constant_hz`, `nqr_frequencies_hz` — C_Q and zero-field transition frequencies.
- `AbinitEFGRecord`, `parse_abinit_efg`, `format_abinit_efg_block` — ABINIT EFG input/output.

9.2 quadrupolar_dft.thermal

- `thermal_quantum_factor`, `bose_occupation`, `mean_square_normal_coordinate`, `wavenumber_to_angular_frequenc`

9.3 quadrupolar_dft.vibrational

- `VibrationalMode`, `efg_curvature_central_difference`, `thermally_averaged_efg`, `efg_temperature_sweep`, `ThermalEFGPoint`.
- `bayer_frequency`, `bayer_slope_hz_per_k`, `fit_bayer_single_mode`, `BayerFit`.

9.4 quadrupolar_dft.finite_displacement / abinit_phonon

- `Crystal`, `PhononMode`, `parse_abinit_structure`, `generate_displacement_jobs`, `abinit_input_with_positions`, `write_jobs`.
- `collect_efg_outputs`, `vibrational_modes_from_collected`, `vibrational_modes_from_efg`, `efg_tensor_from_rec`
- `phonon_dfpt_input`, `anaddb_input`, `parse_anaddb_modes`, `modes_from_arrays`.

9.5 quadrupolar_dft.strain_coupling

- `generate_strain_jobs`, `write_strain_jobs` — central finite-difference homogeneous strain inputs.
- `collect_strain_derivatives` — parse strained EFG outputs and compute $\partial V_{ij}/\partial \epsilon_\mu$.
- `strain_transition_couplings` — project strain derivatives onto spin-1 quadrupolar transition matrix elements.
- `glycine_static_efg_input_from_cif`, `cif_structure_metadata`, `space_group_is_likely_centrosymmetric` — glycine setup and polymorph guards for piezoelectric work.

9.6 quadrupolar_dft.relax

- `relax_input` (ionic-relaxation input from a static EFG input), `parse_relaxed_structure` (relaxed geometry from the ABINIT output footer), `relaxed_input` (relaxed static EFG input for Stages 1–2).

10 Limitations and Roadmap

Current limitations.

- The $C_{\mathrm{Q}} \leftrightarrow \nu_{\mathrm{Q}}$ scale and several helpers are calibrated for spin 1 and spin 3/2; higher half-integer spins are not yet handled by the simulator-facing conversion.
- The generated DFPT input is a template; PAW DFPT settings, tolerances and k-mesh must be checked for production accuracy.
- The anaddb eigenvector parser targets ABINIT 9.x text output; a `-modes` JSON escape hatch covers other layouts.
- Finite-difference curvatures use the printed EFG precision; large displacement amplitudes or extra output digits improve the second derivative.

Roadmap.

- Full cell relaxation (`optcell`) and a quasi-harmonic thermal-expansion path, building on the internal-coordinate relaxation stage (§5.2); plus AIMD/PIMD snapshot averaging for strongly anharmonic systems such as $NaNO_2$ near its ferroelectric transition.
- A netCDF / phonopy phonon reader to complement the anaddb text parser.